# Palm OS Programming Recipes

**Recipe: Using Gadgets (Prior to Palm OS 3.5)**

If you want to define a custom user interface object—a UI object outside the standard UI objects defined by Palm OS® software—then you will need to use a gadget. Generally, a gadget defines an area of the screen in which you will manage the user's interaction by watching all tap events.

**NOTE:** A newer form of gadgets, called *extended gadgets*, was introduced in Palm OS 3.5. If you can limit yourself to Palm OS 3.5 and later, then read the information on using extended gadgets. The information below describes how to use a gadget for applications prior to Palm OS 3.5.

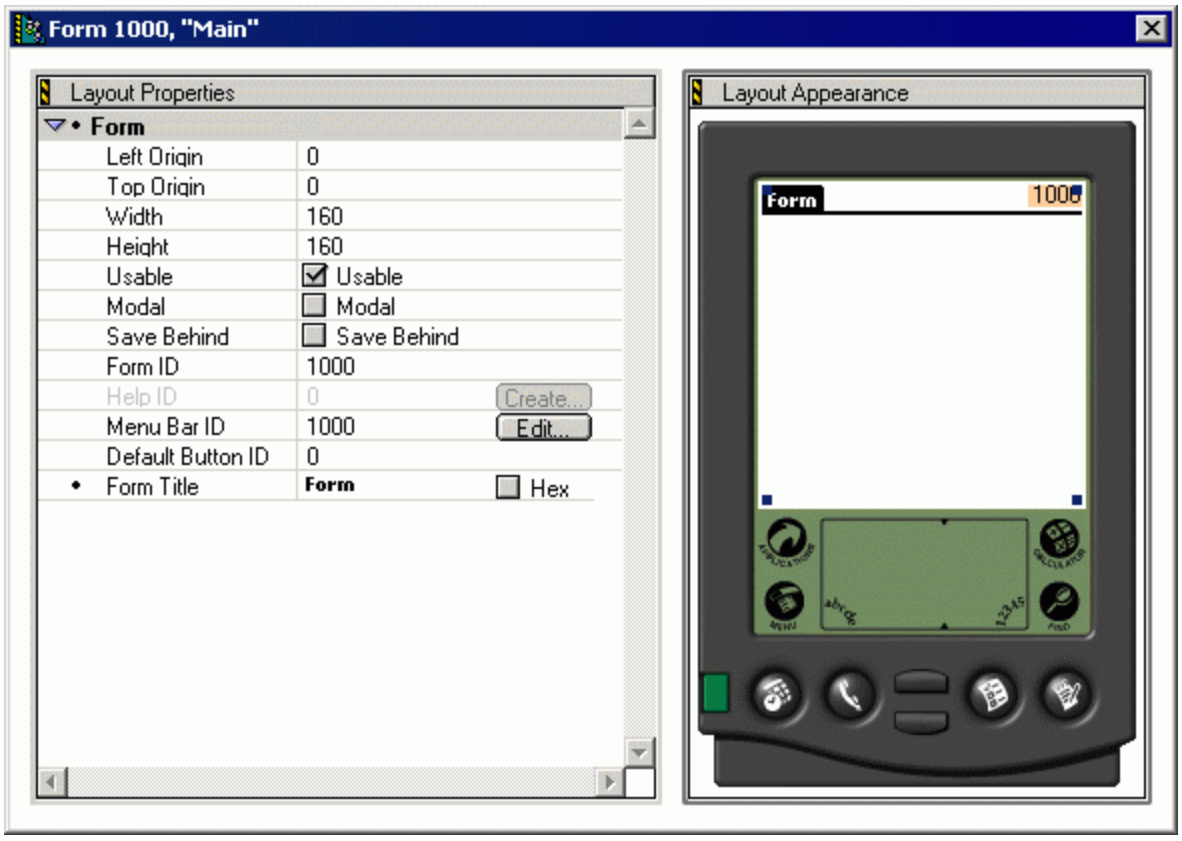
To use a gadget in your application, you need to do the following:

- Use Constructor for Palm OS to add the gadget to the user interface.
- Add code to your application to manage the gadget:
  1. Initialize and free the gadget's data pointer.
  2. Draw, redraw, and erase the gadget.
  3. Handle the user's interaction with the gadget by handling application events.

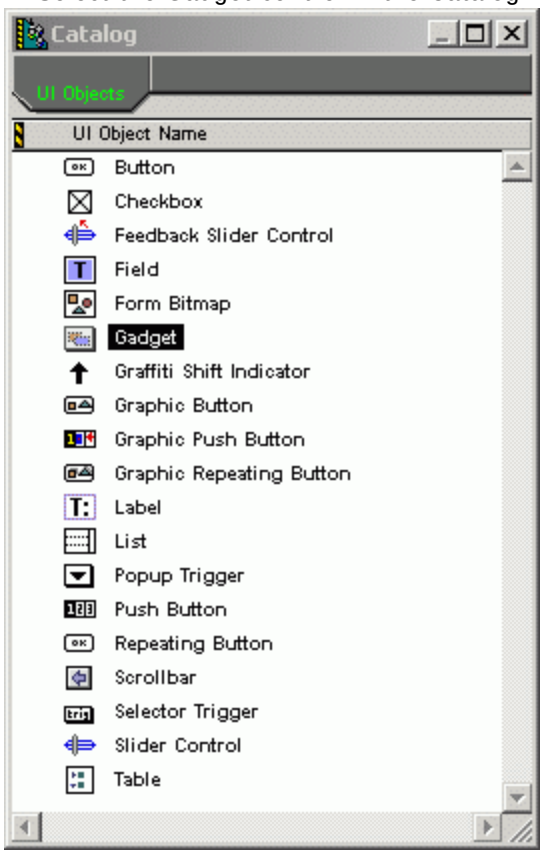
**Add the Gadget to the User Interface**

Use Constructor for Palm OS to add the gadget to your application's user interface.

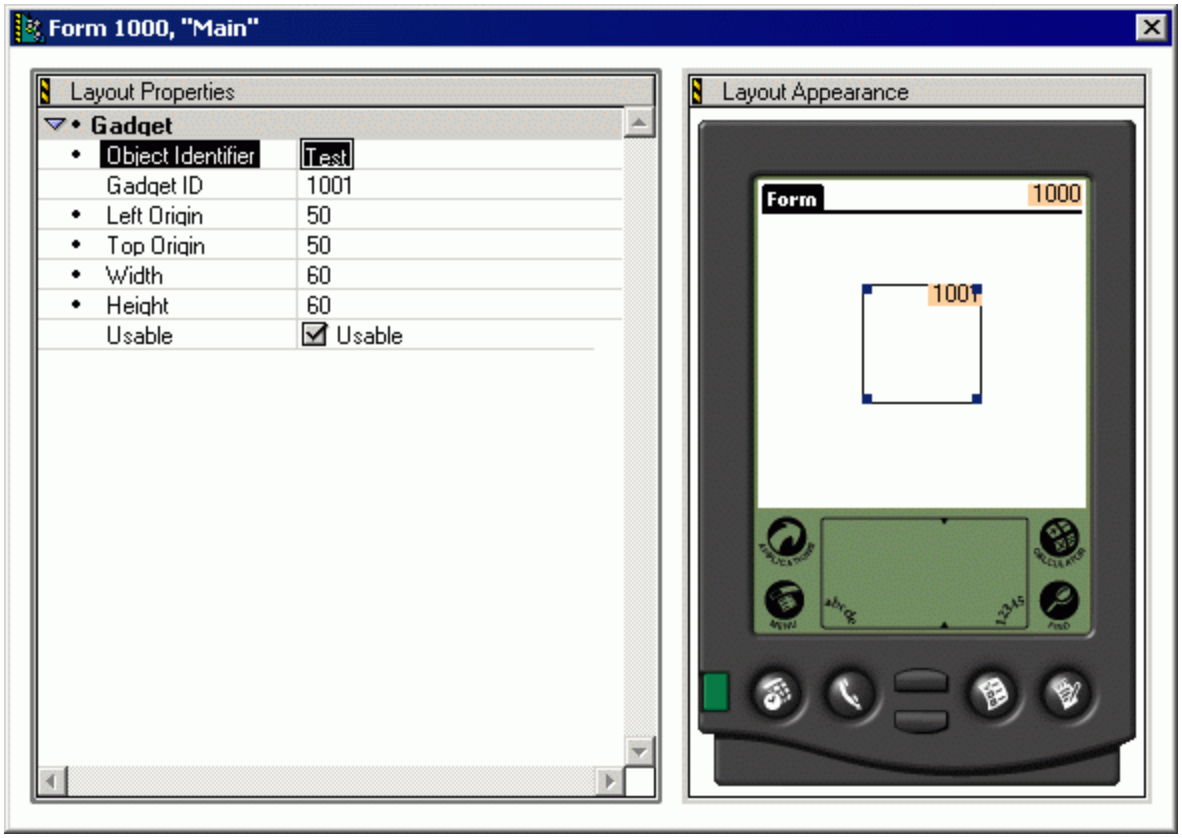
1. Open your application's form in the Form Layout window.



2. Select the Gadget control in the Catalog window.




3. Drag and drop the Gadget control onto the form in the Form Layout window.



4. Update the Gadget control's properties, setting the **Object Identifier**, **Left Origin**, **Top Origin**, **Width**, and **Height**, as appropriate. Leave **Usable** checked.

## Initialize and Free the Gadget's Data Pointer

If you plan on storing data related to the gadget, you will need to allocate and free that data appropriately. Every gadget has an associated void pointer called `data` (defined as `void* data`). Here is how to initialize the gadget's data pointer.

1. First, create a structure to hold the gadget's data.

```
typedef struct
{
   //gadget data here
}GadgetDataType;
```

2. Declare these variables in your form's handler:

```
GadgetDataType* gadgetDataP;
UInt16 gadgetIndex;
```

3. Then, when your application receives the event `frmOpenEvent`, allocate memory for your gadget data and set the gadget's data pointer.

```
case frmOpenEvent:
   frmP = FrmGetActiveForm();
   MainFormInit( frmP);
   FrmDrawForm ( frmP);

   //initialize gadget data
   gadgetIndex = FrmGetObjectIndex(frmP, MainTestGadget);

   gadgetDataP =(GadgetDataType*)
            FrmGetGadgetData(frmP, gadgetIndex);
   if (!gadgetDataP)
   {
      gadgetDataP = (GadgetDataType*)
            MemPtrNew(sizeof(GadgetDataType));

      ErrFatalDisplayIf(!gadgetDataP,
       "Could Not Allocate Memory for Gadget's Data");

      FrmSetGadgetData(frmP, gadgetIndex, (void*)
       gadgetDataP);
   }

   handled = true;
   break;
```

4. The gadget's data will typically be freed in response to the event `frmCloseEvent`. In some cases, you may want to free it elsewhere, such as in the `AppStop` function.

Here's how to free the gadget's data:

```
case frmCloseEvent:
   frmP = FrmGetActiveForm();
   gadgetIndex = FrmGetObjectIndex(frmP, MainTestGadget);

   gadgetDataP = (GadgetDataType*) FrmGetGadgetData(frmP,
             gadgetIndex);
   MemPtrFree((MemPtr) gadgetDataP);
   //remember to leave this as false so the
   //system will finish
   //the deletion of the form
   handled = false;
   break;
```

**Draw, Redraw, and Erase the Gadget**

When your application's form is initially drawn, you will need to custom draw your gadget. You will also need to redraw the gadget whenever the form is redrawn.

The following example shows how to draw a rectangle boundary around a gadget.

1. Add the following declaration to the form handler:

```
RectangleType bounds;
```

2. Then, in response to the events `frmOpenEvent` and `frmUpdateEvent`, do the custom gadget drawing.

```
//add the same to frmOpenEvent
case frmUpdateEvent:
 frmP = FrmGetActiveForm();
 FrmDrawForm(frmP);
 gadgetIndex = FrmGetObjectIndex(frmP, MainTestGadget);

 //As an example of custom drawing we will
 //draw a rectangle boundary around the gadget
 FrmGetObjectBounds(frmP, gadgetIndex, &bounds);

 WinDrawRectangleFrame(simpleFrame, &bounds);
 handled = true;
 break;
```

3. Remember to handle the event `frmUpdateEvent` to ensure your gadget is correctly redrawn. For example, if a modal dialog covers part of the form, then the form will receive a `frmUpdateEvent`. In the case where there was not enough free memory to save the bits behind the modal dialog, the gadget will not be correctly displayed unless your application handles the event `frmUpdateEvent`. Also note that debug ROMs will always force the redrawing of the screen in such cases. On debug ROMs, your gadget will never be redrawn unless you handle the event `frmUpdateEvent`.

4. If you use `FrmHideObject` to hide a gadget, you made need to do some custom erasing, depending on your gadget:

    - In the above example, `FrmHideObject` will not erase the rectangle frame that was drawn around the gadget. Your application will have to call `WinEraseRectangleFrame` to do that for you.

    - Before doing any custom drawing or handling of events like `penDownEvent`, you should confirm that the gadget is usable.

**Handle Application Events**

In order for your gadget to react to user interaction, your application will have to handle pen events in the application's form handler.

The following example shows how to track the pen events to catch when a user lifts the stylus while still within the gadget's bounds.

```
case penDownEvent:
   UInt16 ticksPerSec = SysTicksPerSecond();
   Coord newPointX, newPointY;
   Boolean isPenDown = true;
   frmP = FrmGetActiveForm();
   gadgetIndex = FrmGetObjectIndex(frmP, MainTestGadget);
   FrmGetObjectBounds(frmP, gadgetIndex, &bounds);

   if (RctPtInRectangle (eventP->screenX, eventP->screenY ,
            &bounds))
   {
      //there was a tap in the gadget for you to handle

      //track the pen down event
      while (isPenDown)
      {
         EvtGetPen(&newPointX, &newPointY, &isPenDown);

         //save battery life by putting a delay
         //before checking the pen status again
         SysTaskDelay(ticksPerSec / 10);
      }
      if (RctPtInRectangle (newPointX, newPointY, &bounds));
          //the pen up was also in the gadget
   }

   handled = false;
   break;
```